AlgoViz

DSA Visualizer

Task 1 — System Capabilities

No.	Capability	Description
1	Register User	Allows a new user to create account.
2	Authenticate User	Allows registered users to log in and access protected features.
3	Browse Algorithm	Allows users to view available DSA algorithms.
4	Execute Algorithm	Runs the selected algorithm using the user's input
5	Visualize Algorithm	Displays algorithm execution step-by-step
6	View Complexity	Displays the time and space complexity of an algorithm
7	Save Visualization History	Stores previously performed visualizations for later references.
8	Track Learning Progress	Records algorithms practiced or completed by the user.

Task 3 – Service Contract

1. User Service Contract

Operation	Input	Output	Errors
registerUser()	Name, email, password	User registration confirmation	User already exists, invalid input
authenticateUser()	Email, password	Authentication result	Invalid credentials, user not found
getUser()	User information request	User information	User not found, service unavailable

2. Algorithm Service Contract

Operation	Input	Output	Errors
browseAlgorithms()	Optional category/filter	List of available algorithms	No algorithms available
getAlgorithm()	Algorithm name	Algorithm details	Algorithm not found
getComplexity()	Algorithm name	Time and space complexity	Algorithm not found

3. Visualization Service Contract

Operation	Input	Output	Errors
executeAlgorithm()	Algorithm name, input data	Execution result	Algorithm not found, invalid input
visualizeAlgorithm()	Algorithm name, input data	Visualization steps	Algorithm not found, invalid input
getVisualization()	Visualization request	Visualization information	Visualization not found

4. Progress Service contract

Operation	Input	Output	Errors
saveHistory()	User information, visualization information	History saved confirmation	Invalid data, save failed
trackProgress()	User information, progress information	Updated progress	User not found, invalid progress

---

Task 4 - Central Operation Specification

Central Operation: executeAlgorithm()

Service: Visualization Service

1. Inputs

Field	Description
Algorithm Name	Name of the algorithm (e.g., Bubble Sort)
Input Data	Array or list entered by the user
Visualization Speed	Playback speed selected by the user

2. Outputs

Field	Description
Execution Status	Success or Failed
Visualization Steps	Step-by-step execution
Current Data State	Updated array after each operation
Complexity	Time and Space complexity

3. Error Cases

Error	When it Occurs
Algorithm Not Found	Selected algorithm does not exist
Invalid Input	Input data is empty or incorrect
Unsupported Data	Algorithm cannot process the data
Execution Failed	Unexpected execution error

4. Internal Details Hidden from Callers

How the algorithm is implemented in Java.

How visualization steps are generated.

How complexity is calculated internally.

How data is stored or processed by the backend.

Any database or SQL operations.

Task 6 - Final validation table

Service	Property	Status	Issue / Fix
User Service	Reachable	pass	Exposed through API endpoints.
User Service	Self-contained	Pass	Owns user-related data and operations.
User Service	Contract	pass	Provides defined user operations.
User Service	Independent	pass	Can be developed separately.
User Service	Loosely coupled	pass	Other services communicate through APIs.
Algorithm Service	Reachable	pass	Exposed through API endpoints.
Algorithm Service	Self-contained	pass	Owns algorithm information.
Algorithm Service	Contract	pass	Provides algorithm operations.
Algorithm Service	Independent	pass	Algorithm logic is maintained separately.
Algorithm Service	Loosely coupled	pass	Other services use its API rather than internal data.
Visualization Service	Reachable	pass	Exposed through API endpoints.
Visualization Service	Self-contained	pass	Owns visualization sessions and execution steps.
Visualization Service	Contract	pass	Provides visualization operations.
Visualization Service	Independent	partial	Depends on Algorithm Service; keep the dependency through a stable API contract.
Visualization Service	Loosely coupled	partial	Uses Algorithm Service; avoid direct database access and communicate only through APIs.
Progress Service	Reachable	pass	Exposed through API endpoints
Progress Service	Self-contained	pass	Owns progress and history data.
Progress Service	Contract	pass	Provides progress/history operations

Progress Service	Independent	partial	Depends on user, algorithm and visualization references; use service APIs instead of internal implementation
Progress Service	Loosely coupled	partial	Keep cross-service communication through contracts and avoid direct database access.

Improvements:

All services should expose their functionality through APIs.

Each service should own its own data.

Services should communicate through defined contracts.

No service should directly access another service's database.

Changes to one service should not require changes to another service's internal implementation.

Cross-service dependencies should use stable API operations.